🛡️ IDS Hybride - Système de Détection d'Intrusion Réseau

📋 Description

Système Hybride de Détection d'Intrusion Réseau combinant:

- **Moteur Rule-Based**: Détection d'attaques connues (Port Scan, Brute Force, Flood/DoS)
- **Machine Learning Supervisé**: Classification avec Random Forest/XGBoost

🏗️ Architecture

```
ids_hybride_project/
├── app.py                 # Application Flask (Dashboard)
├── config.py              # Configuration
├── preprocess.py          # Prétraitement des données
├── rule_engine.py         # Moteur Rule-Based
├── ml_supervised.py       # Module ML Supervisé
├── hybrid_detector.py     # Intégration Hybride
├── evaluate.py            # Script d'évaluation
├── database_schema.sql    # Script MySQL
├── requirements.txt       # Dépendances
├── data/                  # Dataset UNSW-NB15
│   ├── UNSW_NB15_training-set.csv
│   └── UNSW_NB15_testing-set.csv
├── models/                # Modèles sauvegardés
├── templates/             # Templates HTML
│   ├── base.html
│   ├── dashboard.html
│   └── alerts.html
└── static/                # Fichiers statiques
```

🚀 Installation

1. Créer l'environnement virtuel

```python
python -m venv venv
source venv/bin/activate  # Linux/Mac
# ou
venv\Scripts\activate     # Windows
```

2. Installer les dépendances

```
pip install -r requirements.txt
```

3. Configurer MySQL

```
# Créer la base de données
mysql -u root -p < database_schema.sql

# Modifier config.py avec vos identifiants MySQL
```

4. Télécharger le Dataset UNSW-NB15

Placer les fichiers dans le dossier `data/`:

- `UNSW_NB15_training-set.csv`
- `UNSW_NB15_testing-set.csv`

Source: https://research.unsw.edu.au/projects/unsw-nb15-dataset

💻 Utilisation

Lancer l'application Web

```
python app.py
```

Accéder à: http://localhost:5000

Entraîner et évaluer le modèle

```
python evaluate.py
```

Utilisation en ligne de commande

```
from hybrid_detector import HybridDetector
from config import Config

# Initialiser
detector = HybridDetector(Config())
detector.initialize()

# Analyser un fichier
alerts, predictions, confidence, y_true = detector.analyze('data/test.csv')

# Évaluer
stats = detector.evaluate_system('data/test.csv')
```

📊 Dataset UNSW-NB15

Labels disponibles

Colonne	Description
`label`	0 = Normal, 1 = Attaque
`attack_cat`	Type d'attaque (9 catégories)

Types d'attaques

- Fuzzers
- Analysis
- Backdoors
- DoS
- Exploits
- Generic
- Reconnaissance
- Shellcode
- Worms

⚙️ Configuration

Modifier `config.py` pour:

- Seuils de détection Rule-Based
- Seuil de confiance ML
- Type de modèle (random_forest/xgboost)
- Connexion MySQL

📈 Métriques

Le système évalue:

- **Accuracy**: Prédictions correctes / Total
- **Precision**: Vrais positifs / (Vrais positifs + Faux positifs)
- **Recall**: Vrais positifs / (Vrais positifs + Faux négatifs)
- **F1-Score**: Moyenne harmonique Precision/Recall

🔧 Technologies

- **Python 3.8+**
- **Flask** - Interface Web
- **Scikit-learn** - Machine Learning
- **XGBoost** - Modèle alternatif
- **MySQL** - Base de données
- **Pandas/NumPy** - Traitement des données

📝 Auteur

Projet de Fin d'Études - Spécialité MI

Note: Assurez-vous d'avoir les fichiers de données UNSW-NB15 avant de lancer le système.